```
  performAction(showMessage, 'Button was clicked!'); //
Pass the showMessage function
}
```

- Explanation:
  - `showMessage(String message)` prints a message.
  - `performAction()` receives a function (action) and a message.
  - Inside `performAction()`, the function is executed with the provided message.

e. Why Is This Useful in Flutter?
- Passing functions as arguments is particularly useful in Flutter for handling user interactions.
- For example, buttons in Flutter accept a function as a parameter for the onPressed property.

f. Advanced Example: Returning Functions

```
Function createMultiplier(int factor) {
  return (int number) => number * factor;
}

void main() {
  var doubleIt = createMultiplier(2); // Returns a function
that multiplies by 2
  print(doubleIt(5)); // Output: 10
}
```

- Explanation:
  - `createMultiplier(int factor)` returns a function that multiplies by factor.
  - When `createMultiplier(2)` is called, it returns a function that doubles numbers.
  - Calling `doubleIt(5)` results in 10.

g. Functions in Dart vs. Java
- Key Differences :

Feature	**Dart (Flutter)**	**Java (Android)**
Function Handling	Functions are first-class objects.	Functions are methods inside classes.
Passing Functions	Pass functions as arguments easily.	Typically use interfaces or anonymous classes.
Returning Functions	Can return functions from other functions.	Requires interfaces or lambda expressions.

Ternary Condition Operation

a. What is a Ternary Condition?
- A ternary condition is a shortcut for writing an if-else statement in a single line.
- It checks a condition and quickly decides what to do based on whether it's true or false.
- It makes the code shorter and easier to read when handling simple conditions.

b. Basic Syntax

```
condition ? valueIfTrue : valueIfFalse;
```

- Explanation:
  - condition → The condition being checked (true or false).
  - ? → If condition is true, the value after ? is returned.
  - : → If condition is false, the value after : is returned.

c. Converting If-Else to Ternary Operator

Regular if-else	Ternary Condition

`if (isLoggedIn) {`
`  return Text('Welcome back!');`
`} else {`
`  return Text('Please log in!');`
`}`
	`Text(isLoggedIn ? 'Welcome back!' : 'Please log in!');`

- Explanation:
  - The ternary condition replaces the need for multiple lines of if-else.
  - It directly returns the appropriate value based on isLoggedIn.

d. When Should You Use a Ternary Condition?

- Use ternary conditions when:
  - The condition is simple (choosing between two values).
  - You need a quick decision in a short expression.
  - You want to make the code more readable in UI widgets (e.g., Text, Button).

e. When Should You Use Regular if-else Instead?

- Use a regular if-else when:
  - The condition is complex and has multiple checks.
  - The result requires additional logic before returning a value.
  - The ternary condition makes the code harder to read.
- This is an example when if-else is better than a ternary condition

```
if (userAge >= 18) {
  print('You are an adult.');
} else if (userAge > 13) {
  print('You are a teenager.');
} else {
  print('You are a child.');
}
```

- Explanation:
  - The ternary operator works best with two choices (true or false).
  - When multiple conditions exist (if-else if-else), a standard if-else is clearer.

f. Ternary Condition in Flutter UI Widgets

- Common use case: Show different UI based on a condition.
- Used in UI elements like Text, Container, Button, etc.

- Helps keep UI code clean and short.
- This is an example changing Text Based on a Condition

```
Text(user.isLoggedIn ? 'Welcome, ${user.name}!' : 'Please
log in.');
```

- Explanation:
  - If `user.isLoggedIn` is true, display `"Welcome, {name}!"`.
  - Otherwise, display `"Please log in."`.

Summary of Widget Hierarchy & Advanced Concepts

- Flutter's UI is built using a widget hierarchy (widget tree).
- The hierarchy improves organization, reusability, customization, and performance.
- Flutter efficiently updates only changed widgets, reducing unnecessary re-renders.
- Unlike Java (Android), Flutter keeps layout and logic in one place, simplifying UI management.
- Dart treats functions as first-class objects, allowing them to be stored, passed, and returned.
- This makes it easier to handle user interactions and reusable components in Flutter.
- Compared to Java, Dart offers a more concise and flexible approach to handling functions.
- A ternary condition is a shorthand for if-else that makes code more concise.
- It is useful for simple conditions, especially in UI elements like Text and Button.
- For complex conditions, it's better to use a regular if-else to maintain readability.
- Avoid excessive nesting of ternary conditions, as it can make the code hard to read.